

최적화개론 Project 3 솔루션

3조



미래자동차공학과 2019048440 최윤석

전기공학과 2020090846 김영민

HANYANG UNIVERSITY

I. 서론

최근 인공지능과 강화학습 기술은 다양한 분야에서 빠르게 발전하고 있으며, 그 응용 범위 또한 자율주행, 로봇 제어, 게임 플레이, 스마트 제조 등으로 꾸준히 확장되고 있다. 특히, **강화 학습(Reinforcement Learning, RL)**은 에이전트가 주위 환경과 상호작용하면서 보상을 최대화하는 방향으로 학습하는 방식으로, 복잡한 의사결정 문제 해결에 적합한 접근법으로 주목받고 있다.

본 프로젝트는 **MiniGrid**라는 강화학습 테스트베드 환경을 활용하여, 에이전트가 주어진 미로를 최적의 경로로 탐색하도록 학습하는 과정을 실험하고 분석하는 것을 목표로 한다. MiniGrid는 시각적으로 간단하지만 동작 구조는 강화학습의 핵심 요소를 내포하고 있어, 정책 설계와 학습 알고리즘 비교에 효과적인 환경을 제공한다.

에이전트는 6x6 크기의 격자 미로 내에서 출발 지점에서 도착 지점까지 이동해야 하며, 행동은 **좌회전(Left)**, **우회전(Right)**, **전진(Forward)**의 세 가지로 제한된다. 이러한 설정은 단순히 이동 경로뿐 아니라 방향성을 고려한 행동 계획이 필요하다는 점에서, 전통적인 경로 탐색 문제와 차별화된다.

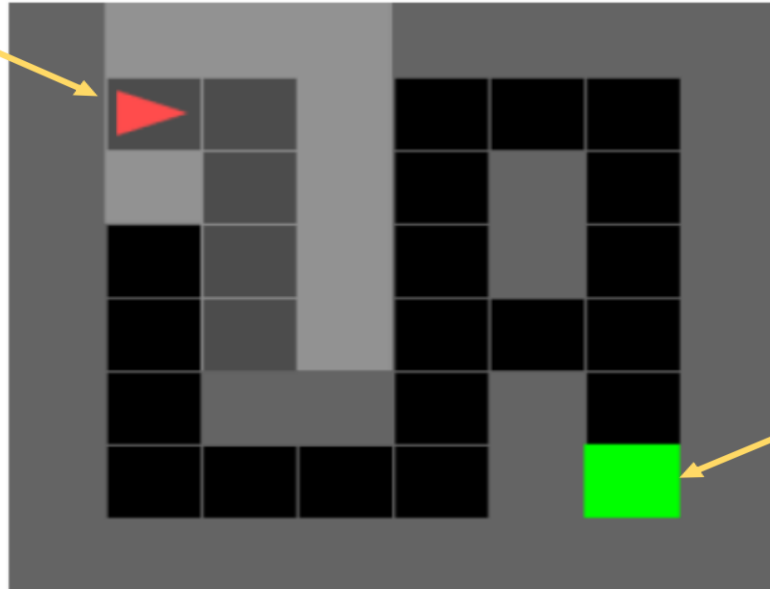
본 프로젝트는 총 세 개의 문제로 구성되어 있다:

- **Problem 1**에서는 MiniGrid 환경 위에 사용자 정의 미로를 생성하고, 환경이 정상적으로 동작하는지 검증한다.
- **Problem 2**에서는 **Q-Learning**과 **SARSA** 알고리즘을 각각 적용하여 미로 탐색 경로를 학습하고, 학습 결과로부터 최적성을 평가한다.
- **Problem 3**에서는 보다 고차원적 함수 근사 방법인 **Deep Q-Network(DQN)**을 적용하여, 신경망 기반의 강화학습 기법이 기존 테이블 기반 기법과 어떻게 다른지를 비교 분석한다.

시뮬레이션은 Python기반으로 구현되었으며, 강화학습 프레임워크로는 **Gymnasium**과 **MiniGrid**, 신경망 구현에는 **PyTorch** 라이브러리를 활용하였다. 또한, 시뮬레이션 학습의 효율성을 위해 클라우드 기반 개발환경인 **Google Colab**을 사용하여 GPU 자원을 확보하고, 반복 학습 및 시각화를 통해 성능을 검증하였다.

II. Problem 1: MiniGrid 기반 6x6 미로 환경 구성

Start



Finish

이 문제는 강화학습 실험에 자주 활용되는 MiniGrid 환경을 기반으로 구성된 과제이며, 에이전트가 제한된 행동 집합을 통해 미로를 탐색하여 목표 지점에 도달하는 시나리오를 다룬다. 환경은 '6행 6열로 구성된 정사각형 격자(Grid) 형태의 미로'로 정의되며, 이 격자 내에는 이동 가능한 공간과 벽으로 구성된 경로가 존재한다. 전체 미로는 컴퓨터 시뮬레이션 상에서 6x6 크기의 고정된 크기로 제공되며, 각 셀은 고유한 좌표를 가진다.

에이전트는 미로의 좌측 상단에 위치한 출발 지점 (1, 1) 에서 탐색을 시작하게 되며, 최종적으로 도달해야 할 목표 지점은 우측 하단의 (6, 6) 위치에 설정되어 있다. 목표 지점은 사용자가 시각적으로 식별할 수 있도록 일반적으로 초록색(Goal tile) 으로 표현된다. 반면, 미로 내부에는 일정 위치에 회색 벽(Wall tile) 이 배치되어 있는데, 이 벽은 에이전트가 통과할 수 없는 장애물 역할을 하여 경로 탐색에 제약을 준다. 벽 외의 공간은 검정색으로 표현되는 aisle(통로) 이며, 에이전트는 이 통로 위에서만 자유롭게 움직일 수 있다. 이 환경에서 에이전트는 완전한 자유도를 가진 것이 아니라, 총 세 가지의 제한된 행동만을 선택할 수 있는 구조로 되어 있다. 각각의 행동은 다음과 같다:

Left: 에이전트가 현재 바라보고 있는 방향을 기준으로 **좌측으로 90도 회전**한다.
Right: 에이전트가 현재 바라보고 있는 방향을 기준으로 **우측으로 90도 회전**한다.
Forward: 에이전트가 현재 바라보고 있는 방향으로 **한 칸 전진**한다. 단, 해당 위치가 벽인 경우 전진하지 못하고 제자리에 머무르게 된다.

이러한 제약된 행동 체계는 전통적인 2차원 격자 기반 탐색 문제와는 다른 특징을 갖는다. 즉, 에이전트는 단순히 상, 하, 좌, 우 네 방향 중 하나를 선택하여 이동할 수 있는 것이 아니라, 현재 바라보고 있는 방향을 기반으로만 움직일 수 있기 때문에 경로 탐색 과정에서 방향성을 고려하는 것이 매우 중요하다. 이로 인해 단순한 최단 거리 탐색 알고리즘으로는 충분하지 않으며, 회전 동작을 최적화하면서 효율적으로 목표에 도달할 수 있는 전략을 설계해야 한다.

따라서 본 문제의 핵심 목표는 에이전트가 제한된 행동만으로 장애물을 피하고, 가능한 한 최소한의 단계(최소 이동 횟수 및 방향 전환)로 목표 지점에 도달할 수 있도록 미로 환경을 정의하고 적절한 정책 또는 경로를 학습 또는 설계하는 데 있다. 이 과정은 강화학습 알고리즘의 비교 및 성능 평가에도 매우 적합한 실험 환경을 제공하며, 다양한 알고리즘(Q-learning, SARSA, DQN 등)의 경로 최적화 능력을 테스트하는 데 유용하게 활용된다.

****주요 코드 분석****

```
# 사용자 정의 6x6 미로 환경
class CustomMazeEnv(MiniGridEnv):
    def __init__(self, size=6, render_mode=None):
        self.size = size
        mission_space = Text(max_length=64)

        super().__init__(
            grid_size=size,
            max_steps=4 * size * size,
            agent_view_size=3,
            mission_space=mission_space,
            render_mode=render_mode
        )
```

1) class CustomMazeEnv(MiniGridEnv): 클래스 정의와 상속

▶ MiniGridEnv를 상속하여 사용자 정의 환경 생성.

▶ 미로의 크기(size)와 시각화 모드(render_mode)를 인자로 받음.

2) super().__init__(grid_size=size, max_steps=4 * size * size, ...): 환경 초기화

▶ 미로 크기를 지정하고, 최대 이동 횟수는 $4 \times \text{size}^2$ 로 제한

이유: MiniGrid 환경에서 에이전트가 미로를 탈출하지 못하고 계속 도는 경우를 방지하기 위

해 최대 스텝 수(max_steps)를 설정.

->미로의 크기가 커질수록 이동해야 할 거리도 증가하므로, size²에 비례하게 설정함. 여기에 4를 곱한 이유는 충분한 여유를 줘서 탐색 중인 에이전트가 실패하지 않고 도달할 수 있는 기회를 어느 정도 부여하기 위한 목적임.

▶ agent_view_size=3은 3x3 범위만 관찰 가능하도록 설정.

이유: 에이전트에게 현실적인 제한된 시야를 주어, 탐색 기반의 학습, 일반화된 정책 형성, 그리고 부분 관찰 환경(POMDP) 상황을 실험할 수 있도록 하기 위한 설정. 에이전트는 자기 자신을 기준으로 앞 방향 기준의 3x3 격자만 관찰 가능.

```
self.action_space = gym.spaces.Discrete(3) # 좌, 우, 전진(Three Actions)

def _gen_grid(self, width, height):
    self.grid = Grid(width, height)
    self.grid.wall_rect(0, 0, width, height)
```

3) self.action_space = gym.spaces.Discrete(3): 행동 공간(Action Space) 정의

▶ Discrete(3): 가능한 행동은 세 가지 – Left, Right, Forward.

4) self.grid = Grid(width, height)

self.grid.wall_rect(0, 0, width, height): 미로 구조 생성 (_gen_grid함수)

▶ MiniGrid의 기본 격자 환경을 초기화하고, 미로의 외곽을 벽으로 감싸 외부 탈출을 방지

▶ Maze 환경의 경계 조건을 명확히 설정

```
# 슬라이드 기반 벽 위치 (에이전트 위치와 겹치지 않도록 (1,1)은 제외)
wall_positions = [
    (1, 3), (1, 4),
    (2, 1), (2, 3),
    (3, 3),
    (4, 1), (4, 5),
    (5, 5),
]
for pos in wall_positions:
    self.put_obj(Wall(), *pos)

# 종료 지점 (6,6) → 인덱스 기준 (5,5)
self.put_obj(Goal(), 5, 5)

# 시작 위치 (1,1) → 인덱스 기준 (1,1) ← (1,1)이 이제 비어 있음
self.agent_pos = (1, 1)
self.agent_dir = 0
self.mission = "Reach the green goal square"
```

5) wall_positions = [(1, 3), (1, 4), (2, 1), (2, 3), (3, 3), (4, 1), (4, 5), (5, 5),]

for pos in wall_positions:

```
self.put_obj(Wall(), *pos)
```

▶ 미로 내 벽을 직접 지정하여 수동 배치

▶ 미로 구조가 고정되어 있기 때문에, 학습 알고리즘이 일관된 경로 탐색 문제를 학습할 수 있도록 환경을 고정

6) self.agent_pos = (1, 1)

```
self.agent_dir = 0
```

```
self.put_obj(Goal(), 5, 5)
```

▶ 에이전트를 미로의 시작 위치에 초기화하고, 방향은 오른쪽(동쪽)을 바라보게 설정

▶ 목표 지점은 (6, 6)으로, 이 위치에 "Goal" 객체를 배치하여 종료 조건을 명확히 설정

▶ 학습 목표를 명확히 지정하고 보상 조건을 정의할 기반을 마련함

7) 시각화 코드

```
# 환경 생성 및 리셋
env = CustomMazeEnv(render_mode="rgb_array")
obs, _ = env.reset()

# 이미지 렌더링
image = env.render()

# 출력
plt.imshow(image)
plt.axis("off")
plt.title("6x6 Maze Matching Slide Example")
plt.show()
```

▶ 구현한 환경을 실제로 시각적으로 확인하고, 미로가 제대로 구성되었는지를 검증

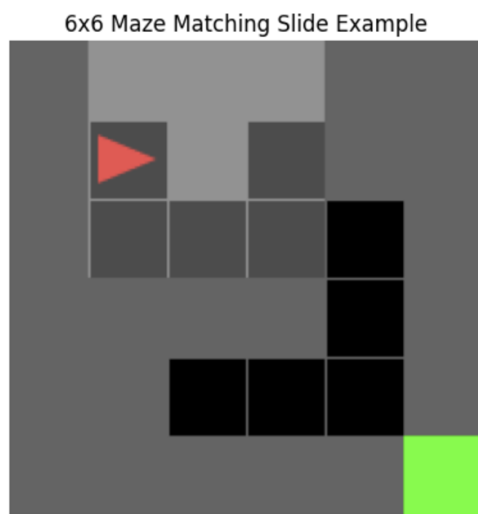
▶ 강화학습 전에 환경 시뮬레이션이 의도대로 구성되었는지 확인하는 필수 단계에 해당함

****코드 실행 결과****

아래 이미지는 시뮬레이션을 통해 환경 구성이 성공적으로 이루어졌는지를 시각적으로 검

증한 결과이다.

☞



시각화 결과를 통해, 에이전트가 탐색해야 할 유효 경로와 장애물의 배치가 명확히 확인된다. 이는 후속 문제에서 사용할 Q-learning, SARSA, DQN 알고리즘의 학습 성능을 비교 실험하기 위한 기초 환경으로 적절히 작동함을 나타낸다.

III. Problem 2: SARSA와 Q-Learning을 이용한 미로 찾기

2-1) Q-Learning을 이용해 문제를 해결하고 최적의 결과인지 판별하기

Q-Learning은 강화학습 분야에서 가장 널리 사용되는 값 기반(value-based) 학습 알고리즘 중 하나로, 에이전트가 환경과의 상호작용을 통해 최적의 행동 정책(optimal policy)을 학습하는 과정에서 사용된다. 본 알고리즘은 환경의 동적 모델을 알 필요 없이 사용할 수 있는 model-free 방식이며, 현재 상태에서 특정 행동을 취할 때 얻을 수 있는 기대 보상의 누적값을 의미하는 Q값(Q-value)을 지속적으로 추정하고 갱신함으로써 최적의 행동 전략을 수립한다.

Q-Learning의 핵심은 벨만 최적 방정식(Bellman Optimality Equation)에 기반한 상태-행동 가치 함수 $Q(s,a)$ 의 반복 갱신에 있다. 에이전트는 매 시간 단계에서 현재 상태 s 에서 가능한 행동 중 하나 a 를 선택하고, 이를 환경에 적용하여 보상 r 과 다음 상태 s' 를 관측한다. 이때 Q값은 다음 식과 같이 갱신된다.

$$Q(s, a) \leftarrow Q(s, a) + \alpha \cdot \left[r + \gamma \cdot \max_{a'} Q(s', a') - Q(s, a) \right]$$

여기서 α 는 학습률(learning rate)로, 새로 관측된 정보가 기존 Q값에 얼마나 반영될지를 결정한다. γ 는 할인율(discount factor)로, 미래 보상의 현재 가치에 대한 중요도를 조절하는 계수이다. $\max_{a'} Q(s', a')$ 는 다음 상태 s' 에서 가능한 모든 행동 중 가장 큰 Q값을 의미하며, 이를 통해 향후 기대 보상의 최댓값을 추정한다. 이러한 방식은 Q-Learning이 off-policy 알고리즘이라는 점을 잘 보여주며, 에이전트는 실제로 어떤 행동을 취했는지에 관계없이 최적의 정책을 기준으로 학습을 수행한다.

****주요 코드 분석****

1)Epsilon Greedy Algorithm

```
# ===== Epsilon-Greedy =====
def eps_greedy(Q_values, state, epsilon=1.0):
    if random.random() < epsilon:
        return random.randint(0, 2) # 3개 행동 중 무작위
    else:
        return int(np.argmax(Q_values[state]))
```

Q-Learning의 특징 중 하나는 탐험과 이용을 균형적으로 수행한다. 이를 위해 Epsilon greedy

코드를 사용하였고 epsilon의 확률로 랜덤하게 행동을 수행하고 나머지 확률로 Q-value가 가장 높은 행동을 선택하는 것이다.

2) Q-Learning 학습 알고리즘

```
# ===== Q-Learning =====
def run(env, Q_values, n_episodes, epsilon, epsilon_decay, gamma, lr):
    total_rewards = []

    for ep in tqdm(range(n_episodes)):
        obs, info = env.reset()
        state = (env.agent_pos[0], env.agent_pos[1], env.agent_dir)
        done = False
        ep_reward = 0

        while not done:
            action = eps_greedy(Q_values, state, epsilon)
            next_obs, reward, terminated, truncated, info = env.step(action)
            next_state = (env.agent_pos[0], env.agent_pos[1], env.agent_dir)
            done = terminated or truncated

            best_next_q = np.max(Q_values[next_state])
            td_target = reward + gamma * best_next_q
            td_error = td_target - Q_values[state][action]
            Q_values[state][action] += lr * td_error

            state = next_state
            ep_reward += reward

        epsilon *= epsilon_decay
        total_rewards.append(ep_reward)

    return Q_values, total_rewards
```

앞서 언급한 벨만 방정식을 기반으로 다음 상태의 Q-value 중 최댓값을 이용해 현재 상태의 Q-value를 갱신한다. 이때 TD 오차만큼 기존의 Q값을 조정하는 방식을 사용하였고 학습률과 할인율을 조정하여 알고리즘의 안정성과 정확도를 조절할 수 있다. 여기서 학습률은 얼마나 새로 관측한 값에 민감하게 반응할 것인지를 나타내는 척도이고, 할인율은 미래 보상의 현재 가치에 대한 중요도를 조절하는 계수이다.

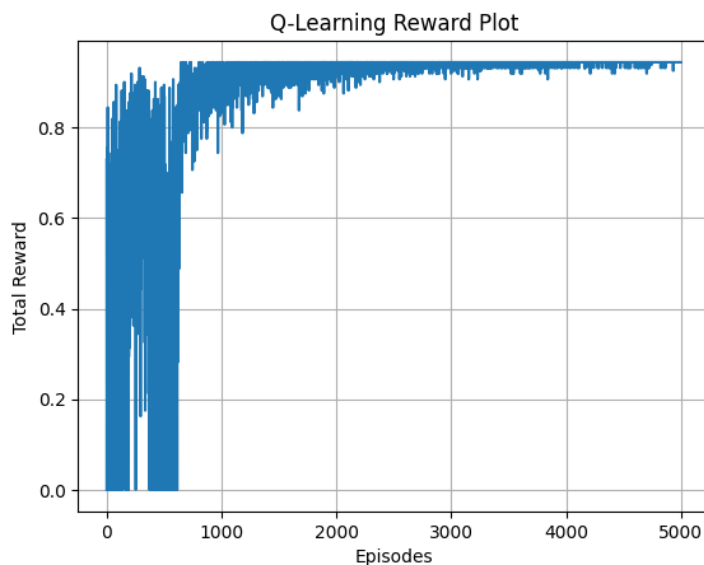
3) 학습 파라미터 설정

```
# ===== 학습 실행 =====
env = SimpleEnv(render_mode="rgb_array")
Q_values = np.zeros((env.width, env.height, 4, env.action_space.n)) # (x, y, dir, action)

Q_values, rewards = run(
    env,
    Q_values,
    n_episodes=5000,
    epsilon=1.0,
    epsilon_decay=0.999, # 더 천천히 감소
    gamma=0.99,
    lr=0.1
)
```

에피소드 수를 크게 함으로써 충분한 반복을 통해 수렴하도록 하고 epsilon_decay를 도입해 점차 무작위 행동을 줄여나간다.

****코드 실행 결과****



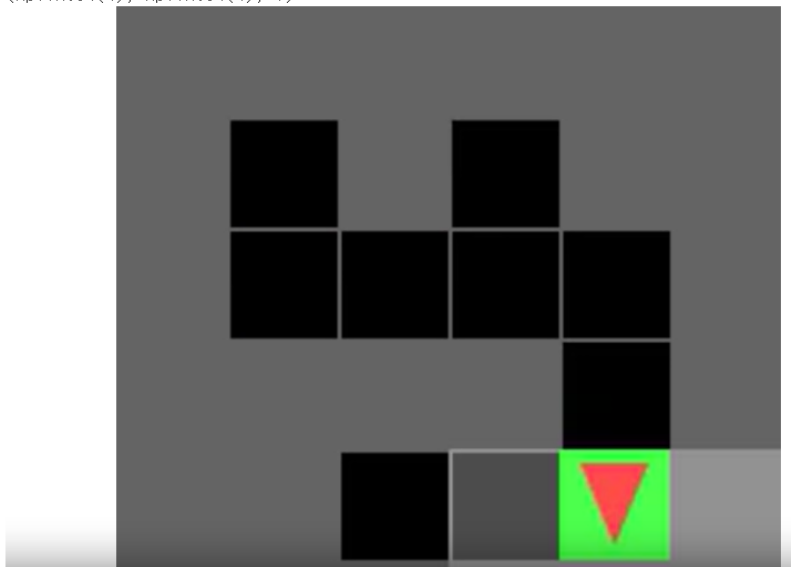
코드를 실행한 결과 초기 200회 미만의 학습 구간에서는 보상이 0에 가까운 경우가 빈번하게 관찰되며, 이는 에이전트가 아직 환경 구조를 학습하지 못해 목표 지점까지 도달하지 못했음을 시사한다. 이 구간에서는 무작위적인 탐험이 주를 이루기 때문에 보상의 변동 폭도 크고 불안정한 양상을 보인다. 하지만 약 300~500 에피소드 이후부터는 보상 값이 0.8 이상으로 빠르게 상승하며, 이후에는 점차 안정적인 수렴 경향을 나타낸다. 특히 1,000 에피소드 이후에는 대부분의 에피소드에서 총 보상이 거의 1에 근접하는 수준을 유지하고 있으며, 이는

에이전트가 일관적으로 목표 지점에 도달할 수 있는 정책을 학습했음을 의미한다.

또한, 수렴 이후에도 일부 에피소드에서 소폭의 보상 감소가 관찰되는데, 이는 ϵ -greedy 정책 특성상 소량의 탐험이 계속 수행되기 때문이며, 전체적인 정책 품질에는 큰 영향을 주지 않는 수준이다.

이러한 학습 곡선은 Q-Learning 알고리즘이 환경 내의 상태 전이와 보상 구조를 성공적으로 파악하고, 최적 또는 준최적의 정책을 효과적으로 학습했음을 보여준다. 특히 ϵ 값을 점진적으로 감소시키며 탐험에서 이용으로 전환하는 전략이 안정적인 수렴을 유도하는 데 크게 기여했음을 확인할 수 있다.

```
✅ State trajectory (x, y, direction):  
(1, 1, 0)  
(1, 1, 1)  
(np.int64(1), np.int64(2), 1)  
(np.int64(1), np.int64(2), 0)  
(np.int64(2), np.int64(2), 0)  
(np.int64(3), np.int64(2), 0)  
(np.int64(4), np.int64(2), 0)  
(np.int64(4), np.int64(2), 1)  
(np.int64(4), np.int64(3), 1)  
(np.int64(4), np.int64(4), 1)
```



시각화 결과를 통해 Q-Learning 알고리즘이 잘 수행된 것을 확인할 수 있다.

2-2) **SARSA**를 이용해 문제를 해결하고 최적의 결과인지 판별하기

SARSA는 강화학습에서 사용되는 on-policy TD 학습 알고리즘으로 에이전트가 자신이 실제로 취한 행동을 기반으로 Q-value를 갱신한다는 점에서 Q-learning과 구분된다. 업데이트 요소는 현재 상태, 현재 행동, 보상, 다음상태, 다음 행동 5가지로 수식으로 표현하면 다음과 같다.

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \cdot [R_{t+1} + \gamma \cdot Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)]$$

SARSA는 on-policy 방식이므로 탐험을 포함한 행동 정책의 영향을 직접적으로 반영한다. 이로 인해 Q-learning보다 더 보수적인 정책을 학습할 수 있으며, 불확실성이 높은 환경에서는 안정적인 수렴을 유도하는데 유리하다는 장점이 있다.

****주요 코드 분석****

```
# SARSA 학습 함수
def run_sarsa(env, Q_values, n_episodes, epsilon, epsilon_decay, gamma, lr):
    total_rewards = []
    for _ in tqdm(range(n_episodes)):
        obs, info = env.reset()
        state = (env.agent_pos[0], env.agent_pos[1], env.agent_dir)
        action = eps_greedy(Q_values, state, epsilon)
        done = False
        ep_reward = 0

        while not done:
            next_obs, reward, terminated, truncated, info = env.step(action)
            next_state = (env.agent_pos[0], env.agent_pos[1], env.agent_dir)
            done = terminated or truncated
            next_action = eps_greedy(Q_values, next_state, epsilon)

            # SARSA 업데이트
            td_target = reward + gamma * Q_values[next_state][next_action]
            td_error = td_target - Q_values[state][action]
            Q_values[state][action] += lr * td_error

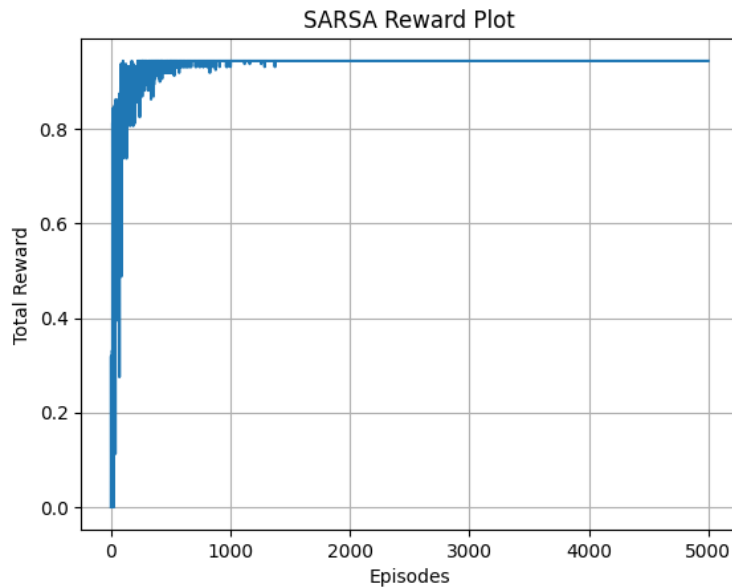
            state = next_state
            action = next_action
            ep_reward += reward

        epsilon *= epsilon_decay
        total_rewards.append(ep_reward)
    return Q_values, total_rewards
```

- ▶ 현재 상태에서 선택된 행동을 환경에 적용하여 다음 상태 및 보상을 얻는다.
- ▶ 환경이 종료 상태에 도달했는지 확인하며, 다음 행동 역시 ϵ -greedy 방식으로 선택된다.
- ▶ 이때 next_action이 곧 SARSA에서 말하는 다음 행동 A_{t+1} 이다.

- ▶ TD Target은 현재 보상과 다음 상태-행동의 Q값을 합한 값이다.
- ▶ TD 오차는 실제 TD 대상값과 현재 Q값의 차이로, 이 값만큼 Q 테이블을 업데이트한다.

****코드 실행 결과****



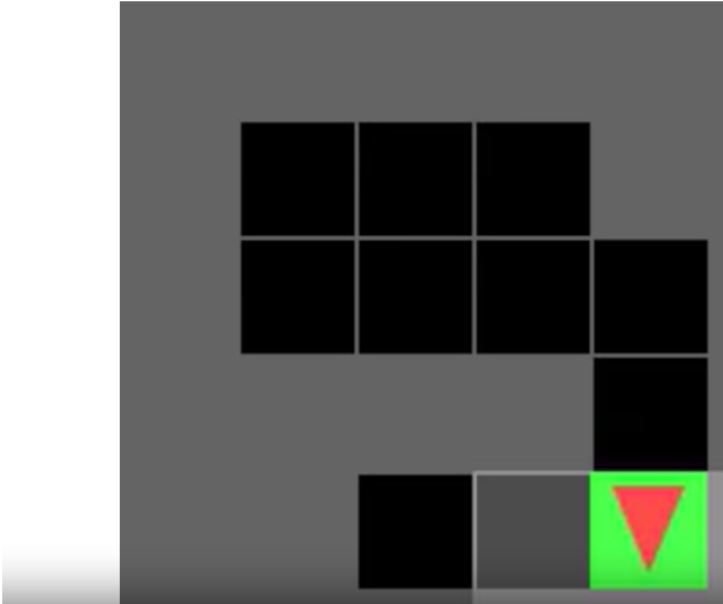
학습 초기 구간(0 ~ 약 200 episode)에서는 보상값이 0에 가까운 값에서 큰 폭으로 진동하는 불안정한 양상을 보인다. 이는 ϵ -greedy 정책에 따라 대부분의 행동이 무작위로 선택되기 때문에, 에이전트가 환경의 구조를 파악하지 못한 상태에서 비효율적인 경로를 선택하거나 목표 도달에 실패하는 경우가 많음을 의미한다.

200 에피소드 이후부터 보상값이 빠르게 상승하며, 대부분의 에피소드에서 보상이 0.9 이상으로 상승하는 급격한 성능 향상 구간이 나타난다. 이는 에이전트가 환경의 상태 전이 구조 및 목표 지점의 위치를 파악하여, 점차 유리한 경로를 선택하기 시작했음을 보여준다. SARSA의 온정책 학습 특성상, Q-value는 실제 정책의 행동을 기반으로 안정적으로 갱신되며, 이로 인해 안정적인 학습이 가능해진다.

약 500 에피소드 이후부터는 거의 모든 에피소드에서 보상값이 1에 근접하거나 동일하게 유지되는 모습이 나타난다. 이는 에이전트가 목표 상태에 일관되게 도달하고 있으며, 정책이 수렴했음을 나타낸다. 이후에도 ϵ 값을 점차 감소시키면서 최적 정책의 재학습 없이도 높은 보상을 지속적으로 획득하고 있다는 점에서, 안정적인 학습 성과를 달성했음을 확인할 수 있다.

✓ State trajectory (x, y, direction):

(1, 1, 0)
(1, 1, 1)
(1, 2, 1)
(1, 2, 0)
(2, 2, 0)
(3, 2, 0)
(4, 2, 0)
(4, 2, 1)
(4, 3, 1)
(4, 4, 1)



시각화 결과를 확인해보면 최적의 결과를 얻었음을 확인할 수 있다.

IV. Problem 3: DQN을 이용한 미로 찾기

DQN(Deep Q-Network)은 강화학습의 대표적인 value 기반 학습 기법 중 하나로, 기존의 Q-Learning에서 사용하는 Q-테이블을 신경망으로 대체한 알고리즘이다. 이는 상태 공간이 연속적이거나 고차원일 경우 Q-테이블을 구성하기 어려운 문제를 극복할 수 있다는 장점이 있다.

기본적으로 DQN은 상태 s 를 입력으로 받아, 가능한 모든 행동 a 에 대한 Q값 $Q(s,a)$ 을 출력하는 회귀 모델로 구성된다. 이후, 다음 식에 기반하여 손실 함수를 최소화하며 학습을 수행한다.

$$\text{Loss} = \left[r + \gamma \cdot \max_{a'} Q_{\text{target}}(s', a') - Q(s, a) \right]^2$$

DQN에서는 또한 학습의 안정성을 높이기 위해 다음과 같은 기법들을 함께 사용한다.

- 1) Target Network 분리 : 일정 주기마다 Q-network의 파라미터를 target network에 복사하여 target 값을 안정화
- 2) Replay Buffer : 과거 경험을 무작위로 샘플링하여 데이터 간 상관성 제거 및 학습 효율 향상
- 3) epsilon-greedy 정책 : 무작위 행동과 최적행동 간 균형을 유지하여 안정성 및 수렴성 확보

****주요 코드 분석****

```
def step(self, action):
    obs, reward, terminated, truncated, info = super().step(action)
    if not terminated:
        reward = -0.01
    if self.agent_pos == self.goal_pos:
        reward = 1.0
    return obs, reward, terminated, truncated, info
```

목표 지점에 도달하면 보상+1, 그렇지 않으면 이동에 대한 페널티(-0.01)를 부여한다. 이로써 빠르게 목적지로 도달하게 유도할 수 있다.

```
# ----- 2. Replay Buffer -----
class ReplayBuffer:
    def __init__(self, capacity):
        self.buffer = deque(maxlen=capacity)
    def push(self, s, a, r, s_, d):
        self.buffer.append((s, a, r, s_, d))
    def sample(self, batch_size):
        s, a, r, s_, d = zip(*random.sample(self.buffer, batch_size))
        return s, a, r, s_, d
    def __len__(self):
        return len(self.buffer)
```

에이전트의 경험을 저장하는 구조로 랜덤 샘플링을 통해 경험 사이의 상관성을 제거하여 안정적인 학습을 유도한다.

```
# ----- 3. DQN Model -----
class DQN(nn.Module):
    def __init__(self, input_shape, num_actions):
        super().__init__()
        self.conv = nn.Sequential(
            nn.Conv2d(input_shape[0], 16, 5, stride=1), nn.ReLU(),
            nn.Conv2d(16, 32, 3, stride=1), nn.ReLU(),
            nn.Conv2d(32, 32, 3, stride=1), nn.ReLU()
        )
        with torch.no_grad():
            dummy = torch.zeros(1, *input_shape)
            conv_out = self.conv(dummy)
            self.flat_dim = int(np.prod(conv_out.size()))
        self.fc = nn.Sequential(
            nn.Linear(self.flat_dim, 256), nn.ReLU(),
            nn.Linear(256, num_actions)
        )
    def forward(self, x):
        x = self.conv(x)
        x = x.view(x.size(0), -1)
        return self.fc(x)
```

상태는 이미지 형태로 주어지므로 CNN을 사용해 Q값을 추정한다. 이때 입력은 전처리된 2D 이미지이며, 출력은 각 행동별 Q값, 즉, left, right, forward의 3개이다.

```
# ----- 6. DQN 학습 루프 -----
def train_dqn(env, model, target_model, buffer, optimizer,
              episodes, batch_size, gamma, epsilon, epsilon_decay, update_freq=20):
    rewards = []
    for ep in tqdm(range(episodes)):
        obs, _ = env.reset()
        state = preprocess(obs).to(device)
        done, total_reward = False, 0

        while not done:
            action = select_action(model, state, epsilon, env.action_space.n)
            obs_, reward, terminated, truncated, _ = env.step(action)
            next_state = preprocess(obs_).to(device)
            done = terminated or truncated
            buffer.push(state, action, reward, next_state, done)
            state = next_state
            total_reward += reward

            if len(buffer) >= batch_size:
                b = buffer.sample(batch_size)
                s = torch.cat(b[0]).to(device)
                a = torch.tensor(b[1]).unsqueeze(1).to(device)
                r = torch.tensor(b[2], dtype=torch.float32).unsqueeze(1).to(device)
                s_ = torch.cat(b[3]).to(device)
                d = torch.tensor(b[4], dtype=torch.float32).unsqueeze(1).to(device)

                q = model(s).gather(1, a)
                with torch.no_grad():
                    max_q = target_model(s_).max(1)[0].unsqueeze(1)
                    target = r + gamma * max_q * (1 - d)

                loss = F.mse_loss(q, target)
                optimizer.zero_grad()
                loss.backward()
                optimizer.step()

            epsilon = max(epsilon * epsilon_decay, 0.01)
            rewards.append(total_reward)

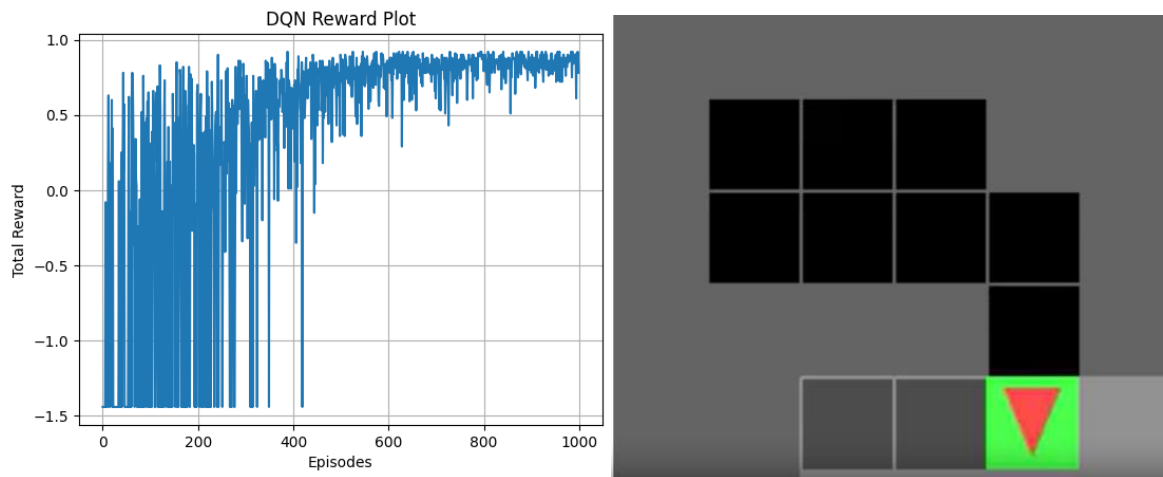
        if ep % update_freq == 0:
            target_model.load_state_dict(model.state_dict())

    return rewards
```

먼저 매 에피소드 시작 시 환경을 초기화할 수 있도록 하고 신경망 입력 형식으로 변환한다.

이후에 현재 상태에서 epsilon-greedy를 기반으로 행동을 선택하고 결과로서 다음 상태를 얻는다. 이 경험이 replaybuffer에 저장되어 이후 학습 데이터로 사용될 수 있는데 때문에 Q-learning과 달리 동일한 경험을 반복적으로 활용하여 표본 효율을 높일 수 있는 장점이 있다.

****코드 실행 결과****



학습 초기에는 보상이 -1.5에서 +1 사이의 넓은 범위로 진동하고 있으며, 특히 음수 구간의 빈도가 매우 높게 나타난다. 이는 DQN이 초기 상태에서 ϵ -greedy 정책을 통해 대부분 무작위적인 행동을 수행하기 때문에, 에이전트가 목표 지점에 도달하지 못하거나 비효율적인 경로를 선택하는 경우가 빈번하다는 것을 의미한다. 이 시점에서는 Replay Buffer에 다양한 행동 정보가 수집되며, 신경망도 아직 충분히 일반화되지 않은 상태이다.

에피소드 수가 약 300을 넘어서면서부터 보상 값이 점차 0 이상에서 안정적으로 유지되기 시작하며, 이후에는 보상의 최댓값인 1.0에 점점 가까워지는 양상을 보인다. 이는 에이전트가 과거의 경험을 바탕으로 보다 효율적인 경로를 예측하고, 목표 지점에 도달하는 빈도가 증가했음을 나타낸다. 또한 이 시점은 ϵ 값이 점차 감소함에 따라 탐험에서 이용으로의 전환이 본격적으로 이루어지는 구간이며, 학습된 Q-network가 타겟 네트워크를 통해 안정적인 피드백을 받아 성능 향상을 주도하게 된다.

에피소드 600 이후부터는 대부분의 보상 값이 0.8 ~ 1.0 사이에서 유지되고 있으며, 급격한 보상 하락이 거의 사라진 것을 볼 수 있다. 이는 에이전트가 목표 지점까지 도달하는 최적 또는 준최적 경로를 안정적으로 학습했다는 것을 시사한다. 또한, Target Network의 주기적 동기화와 Replay Buffer의 샘플링을 통해 학습이 과도하게 진동하거나 불안정해지는 현상도 억제되었음을 확인할 수 있다.

이처럼 DQN은 Q-Learning이나 SARSA보다 학습 초기에 불안정성이 더 클 수 있으나, 충분한 학습이 진행될 경우 더 높은 표현력을 바탕으로 강력한 정책을 학습할 수 있다는 장점이 있다.